
Documento di Specifica dei Requisiti Software

Traccia — Travel Organizer Offline App

Corso di Ingegneria del Software — Gruppo XX

Dicembre 2025

Documento	Specifica dei Requisiti Software (SRS)
Versione	1.0
Stato	Definitivo
Data	Dicembre 2025
Gruppo	XX

Contents

1	Descrizione dell'Applicazione	4
1.1	Scopo dell'applicazione	4
1.2	Funzionalità principali	4
1.3	Attori coinvolti	4
2	Requisiti Funzionali	4
2.1	Business Flow	4
2.1.1	BF-1: Gestione completa dei viaggi	4
2.1.2	BF-2: Gestione delle tappe e dell'itinerario	5
2.1.3	BF-3: Gestione delle attività	5
2.1.4	BF-4: Gestione di checklist e informazioni utili	5
2.1.5	BF-5: Gestione delle spese	5
2.1.6	BF-6: Ricerca, filtri e statistiche	6
2.2	Funzionalità Individuali	6
2.2.1	IF-1.1: Inserimento nuovo viaggio	6
2.2.2	IF-1.2: Modifica dati viaggio	6
2.2.3	IF-1.3: Cancellazione viaggio	7
2.2.4	IF-1.4: Visualizzazione lista viaggi	7
2.2.5	IF-1.5: Ricerca e filtro viaggi	7
2.2.6	IF-2.1: Inserimento nuova tappa	8
2.2.7	IF-2.2: Modifica tappa	8
2.2.8	IF-2.3: Cancellazione tappa	8
2.2.9	IF-2.4: Visualizzazione tappe del viaggio	9
2.2.10	IF-3.1: Inserimento nuova attività	9
2.2.11	IF-3.2: Modifica attività	9
2.2.12	IF-3.3: Cancellazione attività	10
2.2.13	IF-3.4: Visualizzazione e filtro attività	10
2.2.14	IF-4.1: Gestione delle checklist	10
2.2.15	IF-4.2: Gestione delle informazioni utili	11
2.2.16	IF-5.1: Inserimento nuova spesa	11
2.2.17	IF-5.2: Modifica e cancellazione spesa	11
2.2.18	IF-5.3: Visualizzazione e filtro spese	12
2.2.19	IF-6.1: Riepilogo e statistiche	12
2.3	Requisiti relativi ai Dati	12
2.3.1	DF-1.1: Struttura dati Viaggio	12
2.3.2	DF-1.2: Struttura dati Tappa	13
2.3.3	DF-1.3: Struttura dati Attività	13
2.3.4	DF-1.4: Struttura dati Spesa	14
2.3.5	DF-1.5: Struttura dati Checklist e ChecklistItem	15
2.3.6	DF-2.1: Persistenza dei dati	15
2.4	Requisiti dell'Interfaccia Utente	15
2.4.1	UI-1.1: Navigazione principale	15
2.4.2	UI-1.2: Form di inserimento e modifica	16

2.4.3	UI-1.3: Tabelle e liste di visualizzazione dati	16
2.4.4	UI-1.4: Ricerca e filtri	16
2.4.5	UI-1.5: Indicatori visivi di stato	16
2.4.6	UI-1.6: Messaggi di conferma e di errore	17
2.4.7	UI-2.1: Lingua dell'interfaccia	17
3	Requisiti Non Funzionali	17
3.0.1	FC-1: Affidabilità e gestione degli errori	17
3.0.2	FC-2: Integrità dei dati	18
3.0.3	FC-3: Validazione dei dati	18
3.0.4	FC-4: Prestazioni	18
3.0.5	FC-5: Portabilità	19
3.0.6	FC-6: Manutenibilità	19
3.0.7	FC-7: Funzionamento offline	19
4	Scelte Tecnologiche e di Implementazione	19
4.1	Framework e linguaggio	19
4.2	Navigazione	20
4.3	Gestione dello stato	20
4.4	Persistenza locale	20
4.5	Librerie principali	20
4.6	Organizzazione del codice	21
5	Casi d'Uso	21
5.1	Diagrammi dei Casi d'Uso	21
5.2	Descrizione dettagliata dei Casi d'Uso	23
5.2.1	UC-1: Crea Viaggio	23
5.2.2	UC-2: Modifica Viaggio	24
5.2.3	UC-3: Elimina Viaggio	24
5.2.4	UC-4: Visualizza Lista Viaggi	24
5.2.5	UC-5: Cerca e Filtra Viaggi	25
5.2.6	UC-6: Aggiungi Tappa	25
5.2.7	UC-7: Modifica Tappa	26
5.2.8	UC-8: Elimina Tappa	26
5.2.9	UC-9: Visualizza Tappe del Viaggio	26
5.2.10	UC-10: Aggiungi Attività	26
5.2.11	UC-11: Modifica Attività	27
5.2.12	UC-12: Elimina Attività	27
5.2.13	UC-13: Visualizza e Filtra Attività	27
5.2.14	UC-14: Gestisci Checklist	27
5.2.15	UC-15: Gestisci Informazioni Utili	28
5.2.16	UC-16: Aggiungi Spesa	28
5.2.17	UC-17: Modifica Spesa	28
5.2.18	UC-18: Elimina Spesa	29
5.2.19	UC-19: Visualizza e Filtra Spese	29
5.2.20	UC-20: Visualizza Statistiche e Riepilogo	29
5.2.21	UC-21: Timeline Giornaliera (Feature Avanzata)	29

5.2.22 UC-22: Duplica Tappa (Feature Avanzata)	30
6 Tracciabilità	30
6.1 Matrice di Tracciabilità	30
6.2 Legenda	33
A Glossario	33

1 Descrizione dell'Applicazione

1.1 Scopo dell'applicazione

Traccia è un'applicazione mobile sviluppata in React Native che permette di organizzare viaggi in modo completo, mantenendo tutte le informazioni disponibili anche senza connessione internet. L'idea nasce da un'esigenza concreta: quando si è in viaggio la connessione non è sempre garantita, ma la necessità di consultare l'itinerario, aggiornare le spese o spuntare elementi dalla valigia non aspetta. L'app gestisce l'intero ciclo di vita di un viaggio, dalla pianificazione iniziale fino al consuntivo delle spese al rientro.

1.2 Funzionalità principali

- Gestione completa dei viaggi con stati di avanzamento e budget
- Suddivisione del viaggio in tappe o giornate con itinerario ordinato
- Pianificazione di attività associate a tappe, date e luoghi specifici
- Checklist personalizzabili per oggetti, documenti e cose da fare
- Gestione delle spese con distinzione tra previsto ed effettivo
- Sezione statistiche con confronto budget e distribuzione delle spese
- Timeline giornaliera dell'itinerario (feature avanzata)
- Duplicazione di una tappa con tutte le attività associate (feature avanzata)
- Persistenza locale completa: nessun account, nessun server, nessuna rete

1.3 Attori coinvolti

Utente: la persona che pianifica e gestisce i propri viaggi attraverso l'app. Non è previsto alcun sistema di autenticazione né di differenziazione di ruoli. Tutti i dati appartengono al dispositivo su cui l'app è installata.

2 Requisiti Funzionali

2.1 Business Flow

2.1.1 BF-1: Gestione completa dei viaggi

Priorità:	MUST HAVE
------------------	------------------

Rischio tecnico:	Basso
-------------------------	--------------

L'utente deve poter gestire l'intero ciclo di vita dei propri viaggi, dall'inserimento iniziale alla cancellazione definitiva, passando per la modifica e la consultazione. Ogni viaggio

aggrega tutte le informazioni ad esso collegate: tappe, attività, checklist e spese. La lista dei viaggi deve essere consultabile e filtrabile in modo rapido.

2.1.2 BF-2: Gestione delle tappe e dell'itinerario

Priorità:	MUST HAVE
Rischio tecnico:	Basso

L'utente deve poter suddividere un viaggio in tappe o giornate, organizzate in ordine cronologico. Ogni tappa rappresenta l'unità strutturale attorno a cui si aggregano le attività. L'itinerario deve essere consultabile in modo lineare e permettere la navigazione diretta verso le attività di ciascuna tappa.

2.1.3 BF-3: Gestione delle attività

Priorità:	MUST HAVE
Rischio tecnico:	Medio

L'utente deve poter pianificare attività specifiche – visite, pasti, spostamenti, prenotazioni, eventi e altro – associate a un viaggio e, opzionalmente, a una tappa. Il sistema deve permettere di tracciare lo stato di ciascuna attività e di filtrarle per categoria o per giornata.

2.1.4 BF-4: Gestione di checklist e informazioni utili

Priorità:	MUST HAVE
Rischio tecnico:	Basso

L'utente deve poter creare e gestire checklist legate al viaggio per tenere traccia di oggetti da portare, documenti da preparare o attività da completare prima della partenza. Gli elementi delle checklist possono essere marcati come completati e il progresso è visualizzato in modo sintetico. Oltre alle checklist, il sistema supporta note libere e promemoria collegati al viaggio.

2.1.5 BF-5: Gestione delle spese

Priorità:	MUST HAVE
Rischio tecnico:	Basso

L'utente deve poter registrare le spese del viaggio distinguendo tra importi previsti e importi effettivamente sostenuti. Il sistema calcola automaticamente i totali e li confronta con il budget definito in fase di creazione del viaggio. Le spese sono categorizzabili e filtrabili.

2.1.6 BF-6: Ricerca, filtri e statistiche

Priorità: SHOULD HAVE

Rischio tecnico: Basso

Il sistema deve offrire funzionalità di ricerca e filtraggio sui principali dati gestiti, nonché una schermata di riepilogo con statistiche aggregate. Le statistiche comprendono conteggi per stato di viaggio, avanzamento delle attività, distribuzione delle spese per categoria e confronto tra budget previsto e spesa effettiva.

2.2 Funzionalità Individuali

2.2.1 IF-1.1: Inserimento nuovo viaggio

Codice: IF-1.1

Categoria: Funzionalità

Priorità: MUST HAVE

Rischio tecnico: Basso

Casi d'uso correlati: UC-1

Il sistema deve permettere la creazione di un nuovo viaggio specificando i seguenti dati: titolo, destinazione principale, data di inizio, data di fine, descrizione o note generali, stato iniziale, budget previsto, lista dei partecipanti e informazioni utili. Il sistema deve validare che la data di fine non sia antecedente alla data di inizio e che il titolo non sia vuoto. Lo stato iniziale predefinito è *Pianificato*.

2.2.2 IF-1.2: Modifica dati viaggio

Codice: IF-1.2

Categoria: Funzionalità

Priorità: MUST HAVE

Rischio tecnico: Basso

Casi d'uso correlati: UC-2

Il sistema deve permettere la modifica di tutti i campi di un viaggio esistente. Le stesse regole di validazione applicate in fase di inserimento si applicano anche alla modifica. Il sistema deve aggiornare i dati e mostrare un messaggio di conferma al completamento.

2.2.3 IF-1.3: Cancellazione viaggio

Codice:	IF-1.3
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-3

Il sistema deve permettere la cancellazione di un viaggio previa richiesta di conferma esplicita. La cancellazione è a cascata: vengono eliminati automaticamente tutte le tappe, le attività, le checklist e le spese associate al viaggio eliminato.

2.2.4 IF-1.4: Visualizzazione lista viaggi

Codice:	IF-1.4
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-4

Il sistema deve visualizzare l'elenco di tutti i viaggi ordinato per data di inizio in ordine decrescente. Per ciascun viaggio vengono mostrati titolo, destinazione, date di inizio e fine, stato e budget previsto. Nel caso in cui non siano presenti viaggi salvati, il sistema mostra un messaggio informativo con un invito alla creazione del primo viaggio.

2.2.5 IF-1.5: Ricerca e filtro viaggi

Codice:	IF-1.5
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-5

Il sistema deve permettere la ricerca testuale per destinazione (ricerca parziale, case-insensitive) e il filtraggio per stato del viaggio. I risultati si aggiornano in tempo reale durante la digitazione. Se la ricerca non produce risultati, il sistema mostra il messaggio *"Nessun viaggio trovato"*.

2.2.6 IF-2.1: Inserimento nuova tappa

Codice:	IF-2.1
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-6

Il sistema deve permettere l'aggiunta di una tappa a un viaggio specificando: titolo, data, località, descrizione e note. Il titolo è obbligatorio. L'ordine delle tappe nell'itinerario è determinato automaticamente dalla data e può essere modificato manualmente dall'utente.

2.2.7 IF-2.2: Modifica tappa

Codice:	IF-2.2
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-7

Il sistema deve permettere la modifica di tutti i dati di una tappa esistente. Le stesse regole di validazione applicate in fase di inserimento rimangono valide.

2.2.8 IF-2.3: Cancellazione tappa

Codice:	IF-2.3
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-8

Il sistema deve permettere la cancellazione di una tappa previa richiesta di conferma. La cancellazione è a cascata: vengono eliminate automaticamente tutte le attività associate alla tappa rimossa.

2.2.9 IF-2.4: Visualizzazione tappe del viaggio

Codice:	IF-2.4
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-9

Il sistema deve visualizzare le tappe di un viaggio ordinate per data. Per ciascuna tappa vengono mostrati titolo, data, località e numero di attività associate. Selezionando una tappa l'utente accede al dettaglio con le attività collegate.

2.2.10 IF-3.1: Inserimento nuova attività

Codice:	IF-3.1
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Medio
Casi d'uso correlati:	UC-10

Il sistema deve permettere l'aggiunta di un'attività specificando: titolo (obbligatorio), descrizione, data e fascia oraria, luogo, categoria, costo previsto, stato e note. L'attività deve essere collegata a un viaggio e, opzionalmente, a una tappa. Le categorie previste sono: Visita, Pasto, Escursione, Spostamento, Prenotazione, Evento e Altro. Lo stato predefinito è *Da svolgere*. Il costo previsto, se specificato, deve essere un valore numerico maggiore o uguale a zero.

2.2.11 IF-3.2: Modifica attività

Codice:	IF-3.2
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-11

Il sistema deve permettere la modifica di tutti i campi di un'attività esistente, incluso lo stato (Da svolgere, Completata, Annullata).

2.2.12 IF-3.3: Cancellazione attività

Codice:	IF-3.3
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-12

Il sistema deve permettere la cancellazione di un'attività previa richiesta di conferma. La cancellazione di un'attività non comporta l'eliminazione a cascata di altri dati.

2.2.13 IF-3.4: Visualizzazione e filtro attività

Codice:	IF-3.4
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-13

Il sistema deve permettere di visualizzare le attività di un viaggio o di una tappa specifica, con la possibilità di filtrarle per categoria e per stato. Le attività vengono visualizzate ordinate per data e ora.

2.2.14 IF-4.1: Gestione delle checklist

Codice:	IF-4.1
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-14

Il sistema deve permettere di creare una o più checklist per ogni viaggio, di aggiungere, modificare ed eliminare elementi e di marcarli come completati o da completare. Il sistema mostra il progresso di ogni checklist come rapporto tra elementi completati e totali (ad esempio 4/10). La cancellazione di una checklist elimina anche tutti i suoi elementi.

2.2.15 IF-4.2: Gestione delle informazioni utili

Codice:	IF-4.2
Categoria:	Funzionalità
Priorità:	SHOULD HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-15

Il sistema deve permettere di aggiungere, modificare ed eliminare note libere, promemoria e riferimenti a prenotazioni associati al viaggio, come numeri di telefono utili, indirizzi di alloggi o orari di voli e treni.

2.2.16 IF-5.1: Inserimento nuova spesa

Codice:	IF-5.1
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-16

Il sistema deve permettere la registrazione di una spesa specificando: titolo (obbligatorio), importo (obbligatorio, maggiore di zero), categoria, data, tipo (Prevista o Effettiva), metodo di pagamento e note. La spesa deve essere collegata al viaggio e, opzionalmente, a una tappa o a un'attività. Le categorie previste sono: Alloggio, Trasporto, Cibo, Attività, Shopping e Altro.

2.2.17 IF-5.2: Modifica e cancellazione spesa

Codice:	IF-5.2
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-17, UC-18

Il sistema deve permettere la modifica di tutti i campi di una spesa esistente e la sua cancellazione previa richiesta di conferma.

2.2.18 IF-5.3: Visualizzazione e filtro spese

Codice:	IF-5.3
Categoria:	Funzionalità
Priorità:	MUST HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-19

Il sistema deve visualizzare le spese di un viaggio mostrando il totale delle spese previste e il totale delle spese effettive, con indicazione dello scostamento rispetto al budget. L'utente può filtrare le spese per categoria e per tipo (Prevista/Effettiva).

2.2.19 IF-6.1: Riepilogo e statistiche

Codice:	IF-6.1
Categoria:	Funzionalità
Priorità:	SHOULD HAVE
Rischio tecnico:	Basso
Casi d'uso correlati:	UC-20

Il sistema deve fornire una schermata di riepilogo con le seguenti informazioni: numero totale di viaggi per stato, attività pianificate e completate per il viaggio selezionato, spesa totale prevista e spesa totale effettiva con confronto rispetto al budget, distribuzione delle spese per categoria, progresso complessivo delle checklist. Le informazioni numeriche sono affiancate da rappresentazioni grafiche a barre o a torta.

2.3 Requisiti relativi ai Dati

2.3.1 DF-1.1: Struttura dati Viaggio

Codice:	DF-1.1
Categoria:	Dati
Priorità:	MUST HAVE
Rischio tecnico:	Basso

Il sistema deve gestire per ogni viaggio i seguenti dati:

- id (stringa, univoco, generato automaticamente)
- Titolo (stringa, obbligatorio, max 100 caratteri)
- Destinazione (stringa, obbligatorio, max 100 caratteri)

- Data di inizio (data, obbligatorio)
- Data di fine (data, obbligatorio, $\geq$ data di inizio)
- Descrizione (stringa, opzionale, max 500 caratteri)
- Stato (enumerazione: Pianificato, In corso, Completato, Archiviato)
- Budget previsto (decimale, opzionale, ≥ 0)
- Partecipanti (lista di stringhe, opzionale)
- Informazioni utili (testo libero, opzionale)

2.3.2 DF-1.2: Struttura dati Tappa

Codice:	DF-1.2
Categoria:	Dati
Priorità:	MUST HAVE
Rischio tecnico:	Basso

Il sistema deve gestire per ogni tappa i seguenti dati:

- id (stringa, univoco, generato automaticamente)
- tripId (riferimento al viaggio, obbligatorio)
- Titolo (stringa, obbligatorio, max 100 caratteri)
- Data (data, obbligatorio)
- Località (stringa, opzionale, max 100 caratteri)
- Descrizione (stringa, opzionale, max 500 caratteri)
- Ordine (intero, calcolato automaticamente per data, modificabile manualmente)
- Note (stringa, opzionale)

2.3.3 DF-1.3: Struttura dati Attività

Codice:	DF-1.3
Categoria:	Dati
Priorità:	MUST HAVE
Rischio tecnico:	Basso

Il sistema deve gestire per ogni attività i seguenti dati:

- id (stringa, univoco, generato automaticamente)
- tripId (riferimento al viaggio, obbligatorio)

- stageId (riferimento alla tappa, opzionale)
- Titolo (stringa, obbligatorio, max 100 caratteri)
- Descrizione (stringa, opzionale, max 500 caratteri)
- Data e ora (datetime, opzionale)
- Luogo (stringa, opzionale, max 100 caratteri)
- Categoria (enumerazione: Visita, Pasto, Escursione, Spostamento, Prenotazione, Evento, Altro)
- Costo previsto (decimale, opzionale, ≥ 0)
- Stato (enumerazione: Da svolgere, Completata, Annullata)
- Note (stringa, opzionale)

2.3.4 DF-1.4: Struttura dati Spesa

Codice:	DF-1.4
Categoria:	Dati
Priorità:	MUST HAVE
Rischio tecnico:	Basso

Il sistema deve gestire per ogni spesa i seguenti dati:

- id (stringa, univoco, generato automaticamente)
- tripId (riferimento al viaggio, obbligatorio)
- stageId (riferimento alla tappa, opzionale)
- activityId (riferimento all'attività, opzionale)
- Titolo (stringa, obbligatorio, max 100 caratteri)
- Importo (decimale, obbligatorio, > 0)
- Categoria (enumerazione: Alloggio, Trasporto, Cibo, Attività, Shopping, Altro)
- Data (data, obbligatorio)
- Tipo (enumerazione: Prevista, Effettiva)
- Metodo di pagamento (stringa, opzionale)
- Note (stringa, opzionale)

2.3.5 DF-1.5: Struttura dati Checklist e ChecklistItem

Codice:	DF-1.5
Categoria:	Dati
Priorità:	MUST HAVE
Rischio tecnico:	Basso

Il sistema deve gestire i seguenti dati per le checklist:

- id (stringa, univoco)
- tripId (riferimento al viaggio, obbligatorio)
- Titolo (stringa, obbligatorio, max 100 caratteri)

E i seguenti per ciascun elemento della checklist:

- id (stringa, univoco)
- checklistId (riferimento alla checklist, obbligatorio)
- Testo (stringa, obbligatorio, max 200 caratteri)
- Completato (booleano, valore predefinito: false)

2.3.6 DF-2.1: Persistenza dei dati

Codice:	DF-2.1
Categoria:	Dati
Priorità:	MUST HAVE
Rischio tecnico:	Medio

Tutti i dati gestiti dall'app – viaggi, tappe, attività, checklist e spese – devono essere salvati in modo persistente sul dispositivo tramite AsyncStorage (per React Native) o Hive/Isar (per Flutter). I dati vengono caricati all'avvio dell'app e salvati automaticamente dopo ogni operazione di scrittura. L'app deve funzionare correttamente in assenza di connessione di rete in ogni sua parte.

2.4 Requisiti dell'Interfaccia Utente

2.4.1 UI-1.1: Navigazione principale

Codice:	UI-1.1
Categoria:	Interfaccia
Priorità:	MUST HAVE

L'app deve presentare una navigazione coerente e intuitiva tra le schermate principali, con accesso alla lista dei viaggi dalla schermata iniziale e navigazione gerarchica verso tappe, attività, checklist e spese di ciascun viaggio. La schermata delle statistiche deve essere raggiungibile da qualsiasi punto dell'app.

2.4.2 UI-1.2: Form di inserimento e modifica

Codice: UI-1.2
Categoria: Interfaccia
Priorità: **MUST HAVE**

I form devono presentare campi chiaramente etichettati con input appropriati per tipo (tasto testo, selettore data, campo numerico, menu a tendina). La validazione avviene in tempo reale con messaggi di errore posizionati sotto al campo interessato.

2.4.3 UI-1.3: Tabelle e liste di visualizzazione dati

Codice: UI-1.3
Categoria: Interfaccia
Priorità: **MUST HAVE**

Le liste devono mostrare le informazioni principali di ciascun elemento con opzioni di azione rapida accessibili tramite pressione prolungata o menu contestuale. Ogni schermata deve gestire lo stato vuoto con un messaggio informativo.

2.4.4 UI-1.4: Ricerca e filtri

Codice: UI-1.4
Categoria: Interfaccia
Priorità: **MUST HAVE**

Le schermate di lista che supportano la ricerca devono presentare una barra di ricerca nella parte superiore con aggiornamento in tempo reale. I filtri per categoria o stato sono presentati come chip o menu a tendina sopra la lista.

2.4.5 UI-1.5: Indicatori visivi di stato

Codice: UI-1.5
Categoria: Interfaccia
Priorità: **SHOULD HAVE**

Lo stato delle attività (Da svolgere, Completata, Annullata), delle spese (Prevista, Effettiva) e dei viaggi deve essere evidenziato tramite colori e icone coerenti in tutta l'app. Lo scostamento negativo rispetto al budget deve essere segnalato in rosso.

2.4.6 UI-1.6: Messaggi di conferma e di errore

Codice: UI-1.6
Categoria: Interfaccia
Priorità: **MUST HAVE**

Il sistema deve mostrare un dialogo di conferma prima di ogni operazione distruttiva e messaggi di errore chiari e leggibili in caso di validazione fallita o operazione non consentita.

2.4.7 UI-2.1: Lingua dell'interfaccia

Codice: UI-2.1
Categoria: Interfaccia
Priorità: **SHOULD HAVE**

L'interfaccia utente è interamente in lingua italiana.

3 Requisiti Non Funzionali

3.0.1 FC-1: Affidabilità e gestione degli errori

Codice: FC-1
Categoria: Affidabilità
Priorità: **MUST HAVE**
Rischio tecnico: Medio

Il sistema deve:

- Gestire tutti gli errori derivanti da operazioni asincrone senza causare crash dell'app
- Mostrare messaggi di errore chiari e comprensibili in caso di operazione fallita
- Non perdere dati già salvati in caso di errore su un'operazione successiva

3.0.2 FC-2: Integrità dei dati

Codice:	FC-2
Categoria:	Integrità
Priorità:	MUST HAVE
Rischio tecnico:	Medio

Il sistema deve garantire che:

- La cancellazione di un viaggio rimuova a cascata tutte le entità collegate
- I riferimenti tra entità (stageId in un'attività, tripId in una spesa) siano sempre validi oppure esplicitamente nulli
- Non sia possibile salvare dati che violino i vincoli definiti nei requisiti DF

3.0.3 FC-3: Validazione dei dati

Codice:	FC-3
Categoria:	Qualità dei dati
Priorità:	MUST HAVE
Rischio tecnico:	Basso

Tutti i dati inseriti devono essere validati prima del salvataggio:

- Campi obbligatori: verifica non vuoti
- Range di valori: importo > 0 , costo attività ≥ 0 , data fine $\geq$ data inizio
- Lunghezze massime: rispettare i limiti specificati nei requisiti DF
- Formati: date nel formato corretto

3.0.4 FC-4: Prestazioni

Codice:	FC-4
Categoria:	Prestazioni
Priorità:	SHOULD HAVE
Rischio tecnico:	Basso

Il sistema deve garantire:

- Operazioni CRUD: completamento in meno di 1 secondo
- Rendering di liste fino a 100 elementi: meno di 2 secondi
- Caricamento iniziale dell'app: meno di 3 secondi

3.0.5 FC-5: Portabilità

Codice:	FC-5
Categoria:	Portabilità
Priorità:	SHOULD HAVE
Rischio tecnico:	Basso

L'app deve essere eseguibile su Android (API level 21 e superiori) e su iOS 13 e superiori. In caso di utilizzo di React Native, deve poter essere eseguita sia su emulatore che su dispositivo fisico mediante Expo o React Native CLI.

3.0.6 FC-6: Manutenibilità

Codice:	FC-6
Categoria:	Manutenibilità
Priorità:	SHOULD HAVE
Rischio tecnico:	Basso

Il codice deve seguire i principi di Separation of Concerns e Single Responsibility. I componenti riutilizzabili devono essere estratti in moduli condivisi e non duplicati. La struttura delle cartelle e le scelte architetturelle devono essere documentate nel file README della repository.

3.0.7 FC-7: Funzionamento offline

Codice:	FC-7
Categoria:	Disponibilità
Priorità:	MUST HAVE
Rischio tecnico:	Basso

Tutte le funzionalità devono essere pienamente accessibili in assenza di connessione di rete. Non è previsto alcun backend remoto né servizio esterno di qualsiasi tipo.

4 Scelte Tecnologiche e di Implementazione

4.1 Framework e linguaggio

Il progetto è sviluppato in **React Native** con **TypeScript**. La scelta di React Native rispetto a Flutter è motivata da tre fattori principali. Prima di tutto il gruppo ha già familiarità con JavaScript ed il passaggio a TypeScript comporta una curva di apprendimento più contenuta rispetto a Dart. In secondo luogo l'ecosistema npm offre un numero elevato di librerie mature per le funzionalità richieste dalla traccia, in particolare per la

persistenza locale e la visualizzazione di grafici. Infine Expo semplifica notevolmente il setup iniziale e il testing su dispositivi fisici senza richiedere la configurazione di ambienti nativi.

TypeScript è preferito al JavaScript puro perché la tipizzazione statica riduce significativamente la classe di errori a runtime, soprattutto nella gestione dei dati persistenti dove un campo mancante o di tipo errato può causare comportamenti difficili da debuggare.

4.2 Navigazione

Per la navigazione tra schermate si utilizza **React Navigation 6** (libreria `@react-navigation/native`). La struttura di navigazione è organizzata su due livelli:

- **Tab Navigator** in fondo allo schermo con tre voci principali: Viaggi, Statistiche e Impostazioni
- **Stack Navigator** per ciascun tab, che gestisce la navigazione gerarchica verso i dettagli (viaggio → tappa → attività, ecc.)

I dati vengono passati tra le schermate tramite i parametri della route di React Navigation per oggetti semplici, oppure recuperati direttamente dallo store globale per le schermate che necessitano di dati aggregati.

4.3 Gestione dello stato

La gestione dello stato è affidata a **Zustand**, una libreria leggera che non richiede la quantità di codice boilerplate di Redux e si integra bene con TypeScript. Lo stato è suddiviso in cinque store separati – uno per ciascuna entità principale (viaggi, tappe, attività, checklist, spese) – ognuno con i propri selettori e azioni. Questa suddivisione riduce i re-render inutili e facilita il testing isolato dei singoli moduli di stato.

4.4 Persistenza locale

La persistenza è gestita tramite **AsyncStorage** (`@react-native-async-storage/async-storage`). I dati sono serializzati in JSON e salvati con chiavi gerarchiche del tipo `trips`, `stages:{tripId}`, `activities:{tripId}` e così via. Il layer di persistenza è incapsulato in un servizio dedicato (**StorageService**) che espone metodi asincroni di lettura e scrittura. Gli store Zustand si occupano di chiamare il servizio ad ogni modifica, rendendo la persistenza trasparente per i componenti UI.

Si è valutata l'alternativa di SQLite tramite `expo-sqlite` per query più complesse, ma la struttura dei dati dell'app non richiede join relazionali complessi e AsyncStorage risulta sufficiente per i volumi attesi.

4.5 Librerie principali

- `@react-navigation/native` + `@react-navigation/stack` + `@react-navigation/bottom-tabs`
— navigazione tra schermate

- `zustand` — gestione dello stato globale
- `@react-native-async-storage/async-storage` — persistenza locale
- `react-native-gifted-charts` — grafici per la schermata statistiche (grafici a torta e a barre con animazioni)
- `@react-native-community/datetimepicker` — selettore nativo di date e ore nei form
- `react-native-uuid` — generazione di identificatori univoci per le entità
- `react-native-vector-icons` — icone per la navigazione e gli stati

4.6 Organizzazione del codice

Il progetto segue una struttura a feature-based folders: ciascun dominio (viaggi, tappe, attività, checklist, spese, statistiche) ha la propria cartella con schermate, componenti specifici e logica di business. Il codice condiviso (componenti riutilizzabili, tipi TypeScript, servizi, utilità) è collocato in cartelle dedicate al di fuori delle feature. Questa organizzazione facilita la suddivisione del lavoro tra i membri del gruppo e riduce i conflitti di merge.

```
src/  
  features/  
    trips/      (schermate e componenti dei viaggi)  
    stages/     (schermate e componenti delle tappe)  
    activities/ (schermate e componenti delle attività)  
    checklists/ (schermate e componenti delle checklist)  
    expenses/   (schermate e componenti delle spese)  
    stats/      (schermata statistiche)  
  shared/  
    components/ (componenti UI riutilizzabili)  
    hooks/      (custom hook condivisi)  
    store/      (store Zustand)  
    services/   (StorageService e altri servizi)  
    types/      (definizioni TypeScript delle entità)  
    utils/      (funzioni di utilità)  
  navigation/   (configurazione React Navigation)
```

5 Casi d'Uso

5.1 Diagrammi dei Casi d'Uso

I casi d'uso sono organizzati in tre diagrammi tematici per mantenere la leggibilità.

[Inserire qui il Diagramma UC — Gruppo 1: Gestione Viaggi]

Attore: Utente

UC-1 Crea Viaggio UC-2 Modifica Viaggio UC-3 Elimina Viaggio
UC-4 Visualizza Lista Viaggi UC-5 Cerca e Filtra Viaggi

Figura 1: Diagramma dei casi d'uso – Gestione Viaggi

[Inserire qui il Diagramma UC — Gruppo 2: Tappe, Attività e Checklist]

Attore: Utente

UC-6 Aggiungi Tappa UC-7 Modifica Tappa UC-8 Elimina Tappa UC-9
Visualizza Tappe
UC-10 Aggiungi Attività UC-11 Modifica Attività UC-12 Elimina Attività
UC-13 Filtra Attività
UC-14 Gestisci Checklist UC-15 Gestisci Informazioni Utili
UC-21 Timeline Giornaliera «include» UC-13 UC-22 Duplica Tappa «include»
UC-6

Figura 2: Diagramma dei casi d'uso – Tappe, Attività e Checklist



Figura 3: Diagramma dei casi d'uso – Spese e Statistiche

5.2 Descrizione dettagliata dei Casi d'Uso

5.2.1 UC-1: Crea Viaggio

Attore: Utente

Precondizioni: L'utente si trova nella schermata Lista Viaggi

Postcondizioni: Il nuovo viaggio è salvato e appare nell'elenco

Flusso normale:

1. L'utente seleziona "Nuovo viaggio"
2. Il sistema mostra il form di inserimento con i campi del viaggio
3. L'utente compila i campi obbligatori (titolo, destinazione, date)
4. L'utente conferma l'inserimento
5. Il sistema valida i dati inseriti
6. Il sistema salva il viaggio nello storage locale
7. Il sistema torna alla lista e mostra il viaggio appena creato

Flussi alternativi:

- 5a. Dati non validi (campo obbligatorio vuoto, data fine antecedente a data inizio): il sistema evidenzia i campi con errore e mostra un messaggio descrittivo; l'esecuzione riprende dal passo 3.

5.2.2 UC-2: Modifica Viaggio

Attore: Utente

Precondizioni: Esiste almeno un viaggio; l'utente si trova nel dettaglio viaggio

Postcondizioni: I dati del viaggio sono aggiornati

Flusso normale:

1. L'utente seleziona l'opzione "Modifica" dal menu del viaggio
2. Il sistema mostra il form precompilato con i dati attuali
3. L'utente modifica uno o più campi
4. L'utente conferma le modifiche
5. Il sistema valida i nuovi dati
6. Il sistema aggiorna il viaggio nello storage e mostra conferma

Flussi alternativi:

- 5a. Dati non validi: il sistema segnala gli errori, l'esecuzione riprende dal passo 3.

5.2.3 UC-3: Elimina Viaggio

Attore: Utente

Precondizioni: Esiste almeno un viaggio

Postcondizioni: Il viaggio e tutti i dati associati sono eliminati

Flusso normale:

1. L'utente seleziona l'opzione "Elimina" dal menu del viaggio
2. Il sistema mostra un dialogo di conferma con avviso sull'eliminazione a cascata
3. L'utente conferma
4. Il sistema rimuove il viaggio con tutte le tappe, attività, checklist e spese collegate
5. Il sistema torna alla lista viaggi

Flussi alternativi:

- 3a. L'utente annulla: il sistema chiude il dialogo senza apportare modifiche.

5.2.4 UC-4: Visualizza Lista Viaggi

Attore: Utente

Precondizioni: L'utente avvia l'app o naviga alla schermata principale

Postcondizioni: La lista dei viaggi è visualizzata

Flusso normale:

1. Il sistema carica i dati dallo storage locale
2. Il sistema ordina i viaggi per data di inizio in ordine decrescente
3. Il sistema visualizza titolo, destinazione, date e stato per ciascun viaggio
4. Se non esistono viaggi il sistema mostra il messaggio “Nessun viaggio salvato. Crea il tuo primo viaggio!”

5.2.5 UC-5: Cerca e Filtra Viaggi

Attore: Utente

Precondizioni: L’utente si trova nella schermata Lista Viaggi

Postcondizioni: I viaggi corrispondenti ai criteri sono visualizzati

Flusso normale:

1. L’utente inserisce testo nella barra di ricerca oppure seleziona un filtro di stato
2. Il sistema aggiorna la lista in tempo reale mostrando i risultati corrispondenti
3. Se nessun viaggio soddisfa i criteri il sistema mostra “Nessun viaggio trovato”

5.2.6 UC-6: Aggiungi Tappa

Attore: Utente

Precondizioni: L’utente si trova nella sezione Tappe di un viaggio

Postcondizioni: La nuova tappa è salvata e appare nell’itinerario

Flusso normale:

1. L’utente seleziona “Aggiungi tappa”
2. Il sistema mostra il form con i campi: titolo, data, località, descrizione, note
3. L’utente compila i campi obbligatori e conferma
4. Il sistema valida i dati e salva la tappa
5. Il sistema aggiorna l’itinerario ordinando le tappe per data

Flussi alternativi:

- 4a. Titolo vuoto: il sistema segnala l’errore, l’esecuzione riprende dal passo 2.

5.2.7 UC-7: Modifica Tappa

Attore: Utente

Precondizioni: Esiste almeno una tappa

Postcondizioni: I dati della tappa sono aggiornati

Flusso analogo a UC-2 (Modifica Viaggio), applicato alla tappa.

5.2.8 UC-8: Elimina Tappa

Attore: Utente

Precondizioni: Esiste almeno una tappa

Postcondizioni: La tappa e tutte le attività associate sono eliminate

Flusso analogo a UC-3 (Elimina Viaggio). La cancellazione a cascata riguarda le attività collegate alla tappa rimossa.

5.2.9 UC-9: Visualizza Tappe del Viaggio

Attore: Utente

Precondizioni: L'utente si trova nel dettaglio di un viaggio

Postcondizioni: Le tappe del viaggio sono visualizzate in ordine cronologico

Flusso normale:

1. Il sistema recupera le tappe del viaggio dallo storage
2. Il sistema le ordina per data
3. Il sistema visualizza titolo, data, località e numero di attività associate per ciascuna
4. Selezionando una tappa l'utente accede al dettaglio con le attività collegate

5.2.10 UC-10: Aggiungi Attività

Attore: Utente

Precondizioni: L'utente si trova nella sezione Attività di un viaggio o di una tappa

Postcondizioni: La nuova attività è salvata

Flusso normale:

1. L'utente seleziona "Aggiungi attività"
2. Il sistema mostra il form con i campi: titolo, descrizione, data e ora, luogo, categoria, costo previsto, stato e note
3. L'utente compila i campi e conferma

4. Il sistema valida i dati (titolo obbligatorio, costo ≥ 0 se specificato)
5. Il sistema salva l'attività e aggiorna la lista

Flussi alternativi:

- 4a. Titolo vuoto o costo negativo: il sistema segnala l'errore, l'esecuzione riprende dal passo 2.

5.2.11 UC-11: Modifica Attività

Attore: Utente

Precondizioni: Esiste almeno un'attività

Postcondizioni: I dati dell'attività sono aggiornati

Flusso analogo a UC-2. Comprende anche la modifica dello stato (Da svolgere, Completata, Annullata).

5.2.12 UC-12: Elimina Attività

Attore: Utente

Precondizioni: Esiste almeno un'attività

Postcondizioni: L'attività è eliminata

Flusso analogo a UC-3. La cancellazione di un'attività non comporta eliminazione a cascata di altri dati.

5.2.13 UC-13: Visualizza e Filtra Attività

Attore: Utente

Precondizioni: L'utente si trova nella sezione Attività di un viaggio o di una tappa

Postcondizioni: Le attività filtrate sono visualizzate

Flusso normale:

1. Il sistema mostra tutte le attività del viaggio o della tappa, ordinate per data e ora
2. L'utente seleziona un filtro per categoria e/o per stato
3. Il sistema aggiorna la lista in tempo reale

5.2.14 UC-14: Gestisci Checklist

Attore: Utente

Precondizioni: L'utente si trova nella sezione Checklist di un viaggio

Postcondizioni: La checklist è aggiornata e il progresso è salvato

Flusso normale:

1. Il sistema visualizza le checklist del viaggio con il relativo progresso
2. L'utente può creare una nuova checklist inserendo il titolo
3. L'utente aggiunge elementi alla checklist inserendo il testo
4. L'utente può modificare il testo di un elemento, eliminarlo o marcarlo come completato
5. Ogni modifica viene salvata immediatamente nello storage locale

5.2.15 UC-15: Gestisci Informazioni Utili

Attore: Utente

Precondizioni: L'utente si trova nella sezione Info del viaggio

Postcondizioni: Le note sono salvate

L'utente può aggiungere, modificare ed eliminare note libere, promemoria o riferimenti utili collegati al viaggio. Ogni modifica viene salvata immediatamente.

5.2.16 UC-16: Aggiungi Spesa

Attore: Utente

Precondizioni: L'utente si trova nella sezione Spese di un viaggio

Postcondizioni: La nuova spesa è salvata e i totali sono aggiornati

Flusso normale:

1. L'utente seleziona "Aggiungi spesa"
2. Il sistema mostra il form con i campi: titolo, importo, categoria, data, tipo (Pre-vista/Effettiva), metodo di pagamento e note
3. L'utente compila i campi obbligatori e conferma
4. Il sistema valida i dati (titolo obbligatorio, importo > 0) e salva
5. Il sistema aggiorna il totale e la lista delle spese

Flussi alternativi:

- 4a. Importo non valido (≤ 0 o non numerico): il sistema segnala l'errore, l'esecuzione riprende dal passo 2.

5.2.17 UC-17: Modifica Spesa

Attore: Utente

Precondizioni: Esiste almeno una spesa

Postcondizioni: I dati della spesa sono aggiornati

Flusso analogo a UC-2 (Modifica Viaggio).

5.2.18 UC-18: Elimina Spesa

Attore: Utente

Precondizioni: Esiste almeno una spesa

Postcondizioni: La spesa è eliminata e i totali sono aggiornati

Flusso analogo a UC-3 (Elimina Viaggio).

5.2.19 UC-19: Visualizza e Filtra Spese

Attore: Utente

Precondizioni: L'utente si trova nella sezione Spese di un viaggio

Postcondizioni: Le spese filtrate sono visualizzate con i totali aggiornati

Flusso normale:

1. Il sistema mostra tutte le spese del viaggio con i totali previsto ed effettivo
2. L'utente seleziona un filtro per categoria e/o per tipo (Prevista/Effettiva)
3. Il sistema aggiorna la lista e ricalcola i totali in tempo reale

5.2.20 UC-20: Visualizza Statistiche e Riepilogo

Attore: Utente

Precondizioni: L'utente accede alla schermata Statistiche

Postcondizioni: Le statistiche aggregate sono visualizzate

Flusso normale:

1. Il sistema calcola e visualizza il numero totale di viaggi suddivisi per stato
2. Il sistema mostra il numero di attività pianificate e completate
3. Il sistema mostra la spesa totale prevista e quella effettiva con il confronto rispetto al budget
4. Il sistema visualizza la distribuzione delle spese per categoria tramite grafico
5. Il sistema mostra il progresso complessivo delle checklist
6. I dati vengono ricalcolati ad ogni accesso alla schermata

5.2.21 UC-21: Timeline Giornaliera (Feature Avanzata)

Attore: Utente

Precondizioni: L'utente si trova nel dettaglio di una tappa

Postcondizioni: Le attività della tappa sono visualizzate in una timeline oraria

Flusso normale:

1. L'utente seleziona la vista "Timeline" dalla schermata della tappa
2. Il sistema visualizza le attività con orario disposte verticalmente su una scala temporale dalle ore di inizio della prima attività alle ore di fine dell'ultima
3. Le attività prive di orario sono mostrate in una sezione separata "Non pianificate"
4. Selezionando un'attività nella timeline, l'utente accede al suo dettaglio

5.2.22 UC-22: Duplica Tappa (Feature Avanzata)**Attore:** Utente**Precondizioni:** Esiste almeno una tappa nel viaggio corrente**Postcondizioni:** Una copia della tappa con tutte le sue attività è creata**Flusso normale:**

1. L'utente seleziona "Duplica tappa" dal menu della tappa
2. Il sistema chiede la data da assegnare alla tappa duplicata
3. L'utente inserisce la data e conferma
4. Il sistema crea una copia della tappa con nuovi identificatori univoci e la data selezionata; tutte le attività associate vengono duplicate con lo stato reimpostato a "Da svolgere"
5. La nuova tappa appare nell'itinerario nella posizione corretta per data

Flussi alternativi:

- 3a. Data non inserita: il sistema segnala l'errore, l'esecuzione riprende dal passo 2.

6 Tracciabilità

6.1 Matrice di Tracciabilità

La matrice collega i requisiti ai casi d'uso che li implementano, ai componenti software che li realizzano e agli esempi di test che ne verificano il corretto funzionamento. La matrice viene aggiornata durante lo sviluppo man mano che i componenti vengono definiti e i test scritti.

Requisito	Casi d'Uso	Componenti	Casi di Test (esempi)
BF-1	UC-1 – UC-5	TripListScreen, TripFormScreen, TripDetailScreen, TripStore	testCreaTripValido, testModificaTrip, testEliminaTrip, testFiltroPerStato

Requisito	Casi d'Uso	Componenti	Casi di Test (esempi)
BF-2	UC-6 – UC-9	StageListScreen, StageFormScreen, StageDetailScreen, StageStore	testCreaStageValido, testEliminaStageACascata, testOrdinamentoPerData
BF-3	UC-10 – UC-13	ActivityListScreen, ActivityFormScreen, ActivityStore	testCreaActivity, testModificaStato, testFiltroPerCategoria
BF-4	UC-14 – UC-15	ChecklistScreen, ChecklistDetailScreen, ChecklistStore	testCreaChecklist, testToggleItem, testProgressoChecklist
BF-5	UC-16 – UC-19	ExpenseListScreen, ExpenseFormScreen, ExpenseStore	testCreaExpense, testImportoNonValido, testTotaleSpese
BF-6	UC-5, UC-13, UC-19, UC-20	StatsScreen, SearchService	testFiltroRealTime, testCalcoloStatistiche
IF-1.1	UC-1	TripFormScreen, TripStore	testCreaTripValido, testTitoloVuoto, testDateInvalide
IF-1.2	UC-2	TripFormScreen, TripStore	testModificaTripSuccesso, testDataFineAntecedente
IF-1.3	UC-3	TripDetailScreen, TripStore	testEliminaTrip, testEliminazioneACascata
IF-1.4	UC-4	TripListScreen	testOrdinamentoPerData, testStatoVuoto
IF-1.5	UC-5	TripListScreen, SearchService	testRicercaDestinazione, testFiltroStato, testNessunRisultato
IF-2.1	UC-6	StageFormScreen, StageStore	testCreaStageValido, testTitoloVuoto
IF-2.2	UC-7	StageFormScreen, StageStore	testModificaStage
IF-2.3	UC-8	StageDetailScreen, StageStore	testEliminaStageConAttivita
IF-2.4	UC-9	StageListScreen	testOrdinamentoStage, testNumeroAttivita
IF-3.1	UC-10	ActivityFormScreen, ActivityStore	testCreaActivityValida, testCostoNegativo
IF-3.2	UC-11	ActivityFormScreen, ActivityStore	testModificaStato
IF-3.3	UC-12	ActivityListScreen, ActivityStore	testEliminaActivity

Requisito	Casi d'Uso	Componenti	Casi di Test (esempi)
IF-3.4	UC-13	ActivityListScreen	testFiltroPerStato, testFiltroPerCategoria
IF-4.1	UC-14	ChecklistScreen, ChecklistStore	testCreaChecklist, testProgressoChecklist, testToggleCompletato
IF-4.2	UC-15	TripInfoScreen	testSalvaNotaLibera, testModificaNota
IF-5.1	UC-16	ExpenseFormScreen, ExpenseStore	testCreaExpense, testImportoZero
IF-5.2	UC-17, UC-18	ExpenseFormScreen, ExpenseListScreen	testModificaExpense, testEliminaExpense
IF-5.3	UC-19	ExpenseListScreen	testTotalePrevisto, testTotaleEffettivo, testFiltroCategoria
IF-6.1	UC-20	StatsScreen	testCalcoloStatistiche, testDistribuzionePerCategoria
DF-1.1	UC-1, UC-2, UC-3	Trip (tipo TypeScript), TripStore	testCostruttoreTrip, testValidazioneDate
DF-1.2	UC-6, UC-7, UC-8	Stage (tipo TypeScript), StageStore	testCostruttoreStage
DF-1.3	UC-10, UC-11, UC-12	Activity (tipo TypeScript), ActivityStore	testCostruttoreActivity, testStatoDefault
DF-1.4	UC-16, UC-17, UC-18	Expense (tipo TypeScript), ExpenseStore	testCostruttoreExpense
DF-1.5	UC-14	Checklist, ChecklistItem (tipi TypeScript), ChecklistStore	testToggleItem, testProgressoCalculation
DF-2.1	UC-1 – UC-22	StorageService	testSalvataggioPersistente, testCaricamentoAvvio
FC-1	UC-1 – UC-22	Tutti i componenti	testGestioneEccezione, testNoCrashInputVuoto
FC-2	UC-3, UC-8	TripStore, StageStore	testEliminazioneACascata, testRiferimentoNulla
FC-3	UC-1, UC-6, UC-10, UC-16	Tutti i FormScreen	testCampiObbligatori, testRangeValori
FC-7	UC-1 – UC-22	StorageService	testFunzionamentoOffline

6.2 Legenda

Prefisso	Significato
BF	Business Flow
IF	Funzionalità Individuale
DF	Requisito sui Dati
UI	Requisito di Interfaccia Utente
FC	Fattore di Qualità (requisito non funzionale)
UC	Caso d'Uso

A Glossario

Termine	Definizione
Viaggio	Entità principale dell'app. Rappresenta un piano di viaggio con date, destinazione, budget e dati ad esso collegati.
Tappa	Suddivisione di un viaggio corrispondente a una giornata o a una specifica città o area geografica.
Attività	Evento o impegno pianificato durante il viaggio (visita, pasto, spostamento, ecc.), associato a un viaggio e opzionalmente a una tappa.
Checklist	Lista di elementi da preparare o spuntare prima o durante il viaggio.
Spesa prevista	Costo stimato prima della partenza o dell'attività.
Spesa effettiva	Costo realmente sostenuto durante il viaggio.
Budget	Importo massimo di spesa pianificato per il viaggio.
Timeline	Vista grafica delle attività di una tappa disposta su una scala oraria verticale.
Duplicazione tappa	Feature avanzata che crea una copia di una tappa (con le relative attività) su una nuova data.
Storage locale	Meccanismo di persistenza dei dati sul dispositivo (Async-Storage) che non richiede connessione di rete.
CRUD	Create, Read, Update, Delete – le quattro operazioni fondamentali di gestione dei dati.
Offline-first	Principio progettuale per cui l'app funziona completamente senza connessione di rete.
Store	Modulo di gestione dello stato globale implementato con Zustand.